

Liz Ángel Núñez Torres

Grado en Física

Universidad Internacional de La Rioja

19 de Marzo de 2025

Índice

1. Ejercicio 1: Simulación de un oscilador armónico. 3
1.1. Método númerico. 4
1.2. ¿Cómo afecta el número de puntos en t_eval al error numérico? 5
1.3. ¿Qué representa físicamente la curva de error? ¿Hay algún patrón? 6
1.4. Parámetro ω 8
1.5. Omega (ω) en función del tiempo. 9
1.6. Fuerza de enlace de una molécula de óxigeno O_2. 10

2. Ejercicio 2: Energía del oscilador armónico. 11
2.1. Parámetros de simulación. 12
2.2. Amortiguamiento porporcional a la velocidad. 13

3. Ejercicio 3: Pulso en una cuerda. 14
3.1. Parámetros del pulso inicial. 14
3.2. Modificación del pulso inicial. 15
3.3. Pulso inicial - onda cuadrada. 17

4. Ejercicio 4: Ruptura de la velocidad del sonido. 18
4.1. Velocidad Jet. 18
4.2. Interpretación efecto Doppler. 21
4.3. Posición inicial en el origen de las coordenadas. 22

5. Califiación/Observaciones. 23

1. Ejercicio 1: Simulación de un oscilador armónico.

En clase, hemos visto que un oscilador armónico sigue la ecuación diferencial

$$\frac{d^2x}{dt^2} = -\omega^2 x.$$

Las soluciones armónicas a esta ecuación son del tipo $x(t) = A\cos(\omega t - kx)$. Ahora, queremos resolver numéricamente esta ecuación para un valor particular de ω y vamos a comprobar qué diferencia hay entre la solución numérica y la solución analítica. Para ello, vamos a escribir la ecuación diferencial como

$$\frac{dx}{dt} = v, \quad \frac{dv}{dt} = -\omega^2 x.$$

Esto nos permite resolverla numéricamente usando diferentes métodos de integración. Una vez tenemos calculadas las soluciones numérica y analítica, podemos comparar sus valores para ver si son iguales. Para ello, calculamos el error absoluto, esto es, $x_{\text{analítico}} - x_{\text{numérico}}$.

1. Averigua qué método numérico está utilizando `solve_ivp` por defecto. ¿Cambia el resultado si probamos otros métodos, como Runge-Kutta de orden 3 (`solver='RK23'`) o Dormand-Prince (`solver='DOP853'`)?

2. ¿Cómo afecta el número de puntos en `t_eval` al error numérico? ¿Qué pasa si usamos 100 o 20 puntos en lugar de 1000? ¿Y 10000 en vez de 1000?

3. ¿Qué representa físicamente la curva de error? ¿Hay algún patrón? Discute qué sucede si simulamos tiempos muy grandes.

4. Cambia el parámetro de ω para tal que la simulación corresponda a un oscilador armónico con $k =$ el número correspondiente a la primera letra de tu nombre ($A = 1, B = 2$, ...) y $m =$ el número correspondiente a la primera letra de tu nombre. Debes adaptar el intervalo de tiempo de la simulación para que tu resultado sea lo más claro posible. Adjunta la gráfica de la solución a tu respuesta.

5. Haz que ω sea una función del tiempo, por ejemplo, $\omega(t) = 1 + \Omega\sin(t), \Omega \in (0, 50)$. Comprueba que cuando $\Omega \approx 0$ se recupera un oscilador armónico normal, mientras que para valores altos de Ω el resultado es más exótico.

6. Obtén la constante de fuerza del enlace de una molécula de O_2. Esto puede hacerse sabiendo que la frecuencia fundamental de vibración del oxígeno molecular es aproximadamente $f = 4{,}7 \times 10^{13}$ Hz, y que la masa de cada uno de los oxígenos es de $m = 2{,}66 \times 10^{-26}$ kg. Vamos a suponer que la molécula se comporta como un oscilador armónico simple, con ambos átomos conectados por un resorte ideal. En tal caso, la frecuencia angular es $\omega = \sqrt{\frac{k}{\mu}}$, donde μ es la masa reducida. Una vez tengas k, simula el movimiento de la molécula, ajustando `t_span` y `t_eval`.

1.1. Método número.

El método por defecto de `solve_ivp` es `RK45` (Runge-Kutta de orden 4/5), que es una variante del método de Dormand-Prince. Si no especificas el parámetro `solver`, se usará `RK45`.

¿Cambia el resultado si usamos otros métodos?

- **Método:** `RK45` (Por defecto).

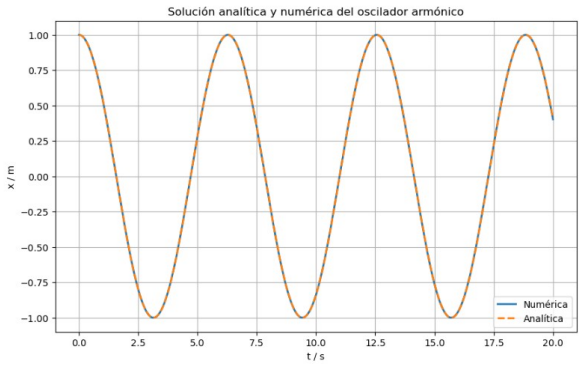


Figura 1: Gráfica del método por defecto.

- **Método:** `RK23` Runge-Kutta de orden 3.

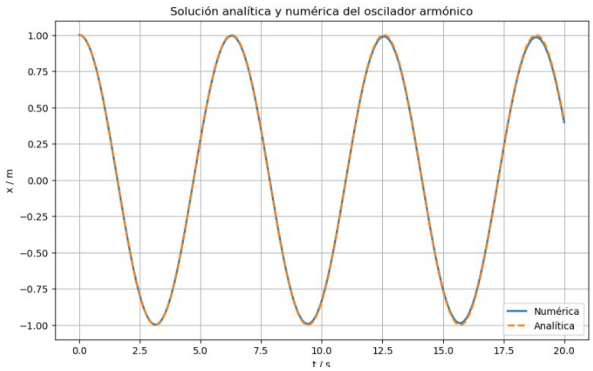


Figura 2: Gráfica del método RK23 de orden 3.

- **Método:** `DOP853` Runge-Kutta de orden 8 .

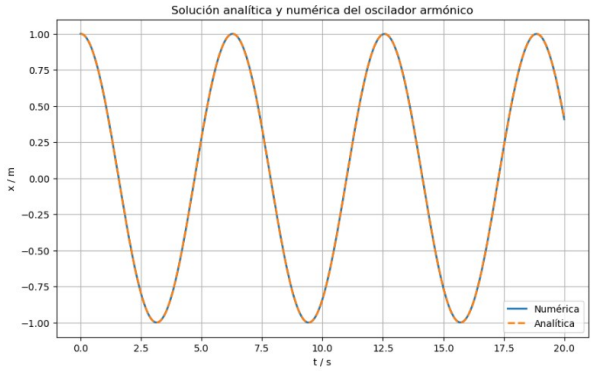


Figura 3: Gráfica del método DOP853 de orden 8.

Como podemos observar, no se aprecia un cambio considerable más allá del suavizado en la gráfica sinusoidal, esto es debido que la solución analítica (**donde si existe un cambio número**) es suave, además el error es muy pequeño del orden de 10^{-6} o menos, lo que ocasione que no sea facilmente perceptible el cambio de método. Cabe resaltar, que en la imagen *"Error absoluto entre la solución numérica y la solución analítica"* **no se presenta ningún cambio más allá de la calidad de las lineas** al igual que en las graficas anteriormente expuestas.

1.2. ¿Cómo afecta el número de puntos en t_eval al error numérico?

Vamos a mostrar como afecta `t_eval` al error numérico con 20 puntos y con 10000 puntos, debido que este enorme intervalo de puntos nos genera una apreciación más solida para argumentar esta variación.

- Error númerico cuando `t_eval` $\Rightarrow$ `nt` $= 20$.

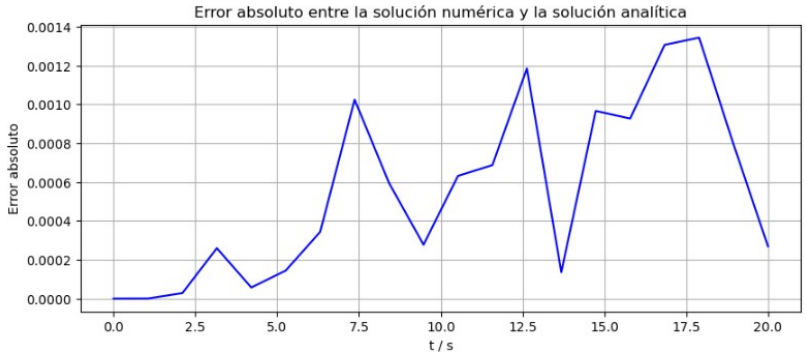


Figura 4: Gráfica "Error absoluto entre la solución numérica y la solución analítico" con 20 puntos.

- Error número cuando `t_eval` $\Rightarrow$ `nt` $= 20$.

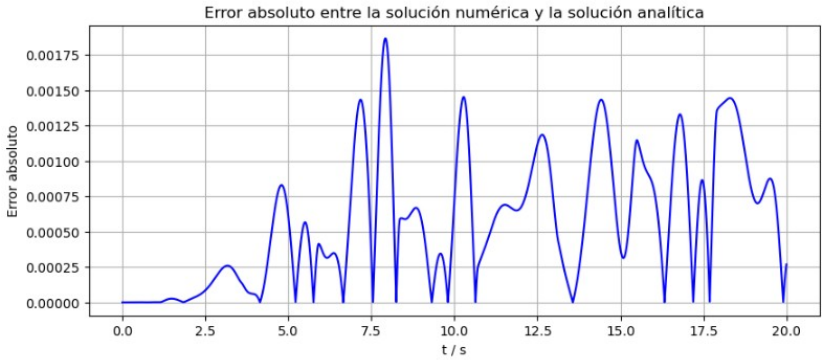


Figura 5: Gráfica "Error absoluto entre la solución numérica y la solución analítico" con 1000 puntos.

Como podemos observar en las imagenes, nuestras gráficas cambian de **escalonada** a más **suave** especialmente en los picos. Esto es debido que al **disminuir los puntos, aumenta el error número** y al **aumentar los puntos, disminuye el error número**; podemos argüir que existe una relación **inversamente proporcional a los puntos.**

1.3. ¿Qué representa físicamente la curva de error? ¿Hay algún patrón?

La curva de error representa la discrepancia entre el método número y la verdadera solución, estos errores ocurren por:

- **Método:** Como hemos visto antes, existen métodos de menor orden, como `RK23` que es orden 3, lo cual afecta a su precisión, por esto, para una solución más exacta, recomendamos usar métodos del orden 8, como lo es `DOP853`.
- **Discretización temporal** Al usar pasos finitos en lugar de infinitamente pequeños (como pudimos constatar en nuestro anterior apartado), se pierde información entre puntos.

Podemos ver en linea de código (`error = np.abs(`$sol.y[0]$ `-` $x_{analytic}$`)`) que mide la diferencia entre la **solución número** (`sol`) y la **solución analítica** ($x_{analytic}$).

Si modificamos el tiempo obtenemos que:

- **Tiempo** `t_span` = 0, 100.

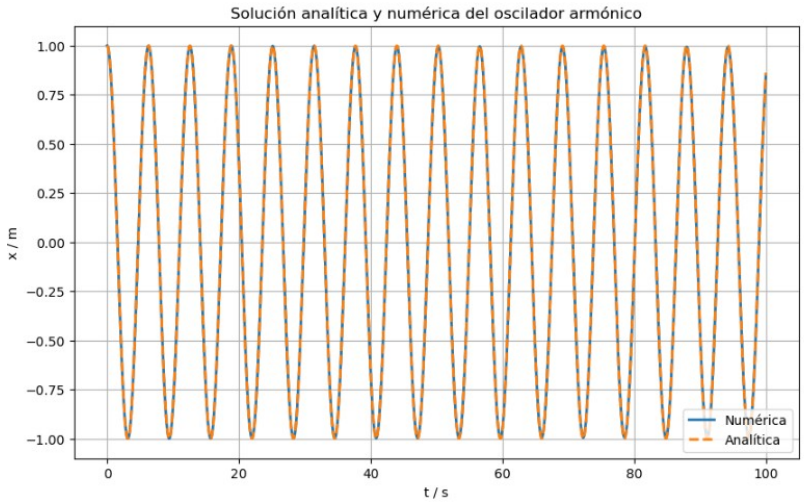


Figura 6: Tiempo 100 con método `DOP853` y `nt` = 1000.

- **Tiempo** `t_span` = 0, 1000.

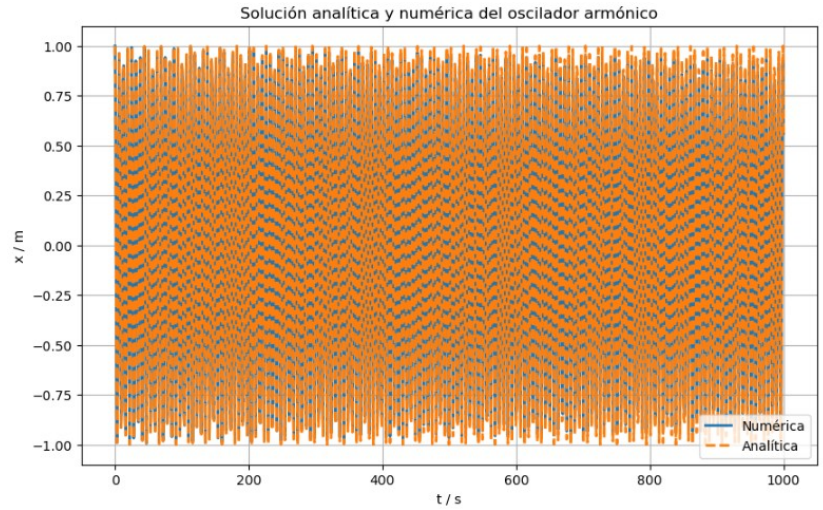


Figura 7: Tiempo 1000 con método `DOP853` y `nt` = 1000.

El **patrón** y conclusión **respecto al tiempo** es que el error absoluto entre la solución númerica y analítica crece con el tiempo como podemos observar en la `Figura 7` con t_span = 1000.

1.4. Parámetro ω .

Modificamos ω donde $\mathbf{L} = 12$.

Ejercicio 1: simulación de un oscilador armónico

```
# establecemos los parámetros.

k = 12       # la letra L = 12.
m = 12       # la letra L = 12.
omega = np.sqrt(12/12)   # frecuencia angular
x0 = 1.0     # posición inicial
v0 = 0.0     # velocidad inicial
```

Figura 8: Sustitución de ω por $\sqrt{\frac{k}{m}}$

Modificamos el tiempo por **dos ciclos completos.**

```
# intervalo de evolución temporal
nt = 5000
t_span = (0, 12.56) # Modificamos para dos ciclos completo.
t_eval = np.linspace(t_span[0], t_span[1], nt)
```

Figura 9: Sustitución del tiempo por 2 ciclos completos.

Grafica con $\omega = \sqrt{\frac{12}{12}}$.

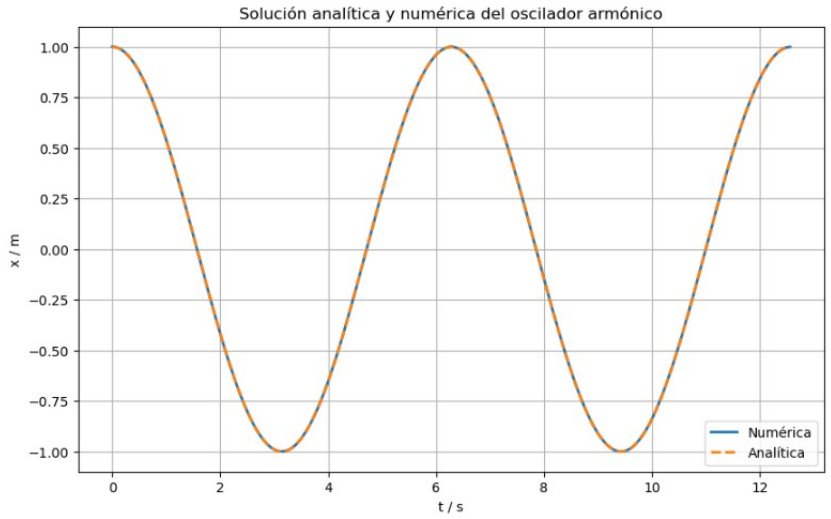


Figura 10: $\omega = 1$ con 2 π ciclos.

Aunque hemos modificado ω esta sigue siendo la misma númerica mente debido que nuestra raíz es igual a 1, al modificar el tiempo podemos encontrar la correlación entre la sólución númerica y analítica sin embargo no presenta un cambio perceptible en la grafica.

1.5. Omega (ω) en función del tiempo.

Para este punto definimos una variable Ω = 0 más la definición de ω en función del tiempo.

```python
Omega = 0  # Creamos la variable que llamararemos en la función deifnida por el tiempo.

# Definimos omega como función del tiempo
def omega_t(t):
    return 1 + Omega * np.sin(t)
```

Figura 11: Omega ω en función del tiempo.

Como podemos obsevar en el código, definimos $\Omega = 0$ lo que genera que nuestra gráfica de la solución analítica y numérica del oscilador armónico **no cambie**.

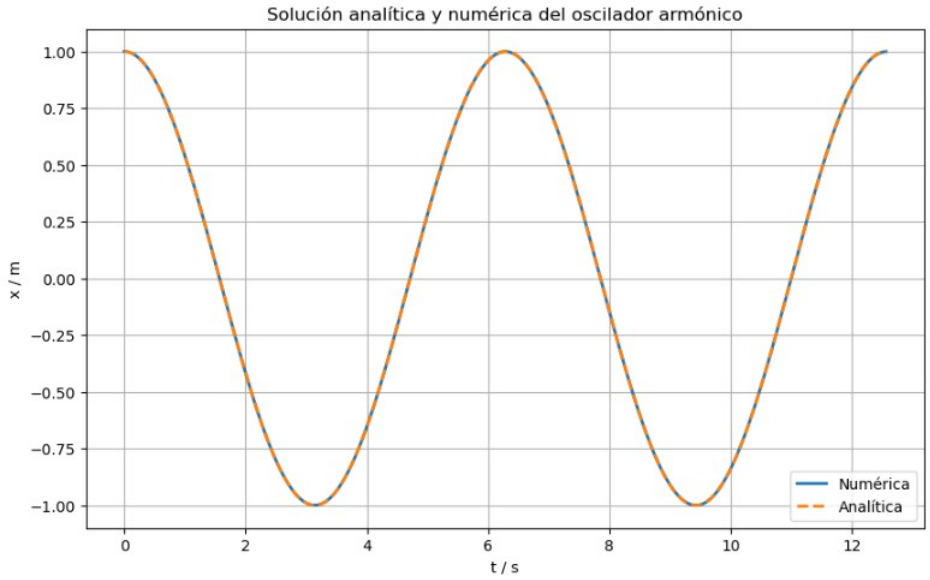


Figura 12: No se obtiene ningún cambio al definir $\Omega = 0$.

Al definir $\Omega > 0$ genera que nuestra gráfica de la solución analítica y numérica del oscilador armónico presente un desface. Esto es porque cambiamos de un **oscilador amónico clásico** donde la frecuencia es constante a un **oscilador paramétrico** con la frecuencia dependiente del tiempo.

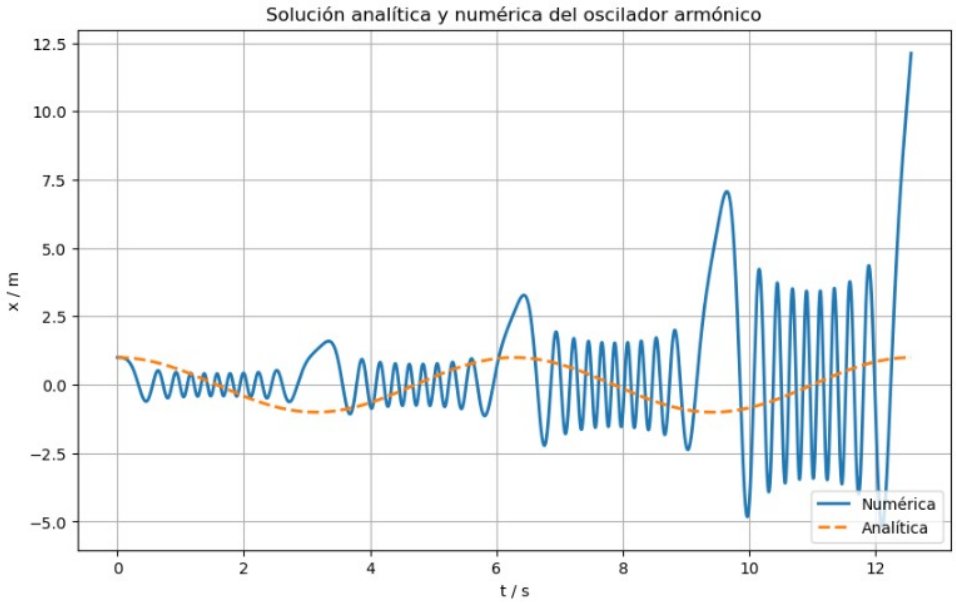


Figura 13: Cuando $\Omega = 30$.

1.6. Fuerza de enlace de una molécula de óxigeno O_2.

Adjuntamos los datos a nuestro código, despejamos ω y obtenemos el valor de k.

```
# establecemos los parámetros.

# Constantes físicas
m = 2.66e-26  # kg
mu = m / 2
f = 4.7e13    # Hz
omega = 2 * np.pi * f
k = mu * omega**2

# Condiciones iniciales
x0 = 1e-11    # desplazamiento pequeño en metros (~0.01 Å)
v0 = 0.0

# Tiempo: queremos observar unos cuantos periodos
T = 1 / f
t_span = (0, 3 * T)
nt = 1000
t_eval = np.linspace(t_span[0], t_span[1], nt)

# Ecuación diferencial
def harmonic_oscillator(t, y):
    x, v = y
    return [v, -omega**2 * x]

# calculamos la solución numérica
sol = solve_ivp(harmonic_oscillator, t_span, [x0, v0], t_eval=t_eval, method ='DOP853')

# y calculamos la solución analítica
x_analytic = x0 * np.cos(omega * t_eval) + (v0 / omega) * np.sin(omega * t_eval)
```

Figura 14: Datos de la molécua O_2 y depseje de k.

Y la gráfica obtenida de la fuerza de enlace es:

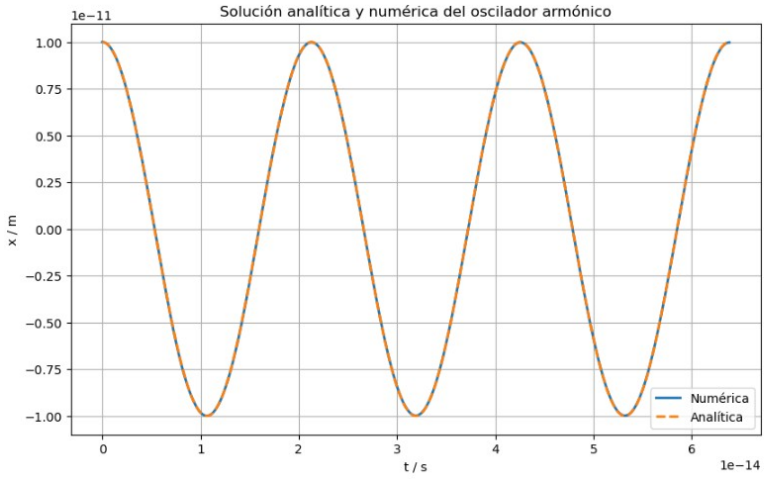


Figura 15: Datos de la molécua O_2 y depseje de k.

2. Ejercicio 2: Energía del oscilador armónico.

Este ejercicio trata del mismo problema que el anterior, pero vamos a enfocarnos en su energía. Además, aprovechamos para ver cómo podemos hacer una animación en Python, la cual nos ayuda a ver con mayor claridad la evolución temporal del sistema.

1. Cambia los parámetros de la simulación a unos valores elegidos por ti y adjunta la nueva figura resultante. ¿Cuál es el valor máximo de la energía cinética? ¿Y de la potencial? ¿Cuándo se alcanzan estos valores? ¿Y de la energía total? ¿Es este el comportamiento que esperabas? Relaciona los máximos de energía cinética y potencial con la velocidad y aceleración.

2. Introducid un amortiguamiento proporcional a la velocidad (resistencia viscosa), lo cual puede ser modelizado tomando $a = -\omega^2 x(t) - \gamma v^2$, con γ una constante arbitraria. Adjunta la figura resultante. Comenta cómo evoluciona la energía en este caso.

2.1. Parámetros de simulación.

Modificamos los parámetros.

```
# Parámetros físicos modificados
omega = 2.0  # Frecuencia angular mayor
x0 = 2.0     # Mayor amplitud inicial
v0 = 5.0     # Velocidad inicial no nula
m = 1.0
k = m * omega**2
```

Figura 16: Parámetros modificados.

La imagen obtenida es:

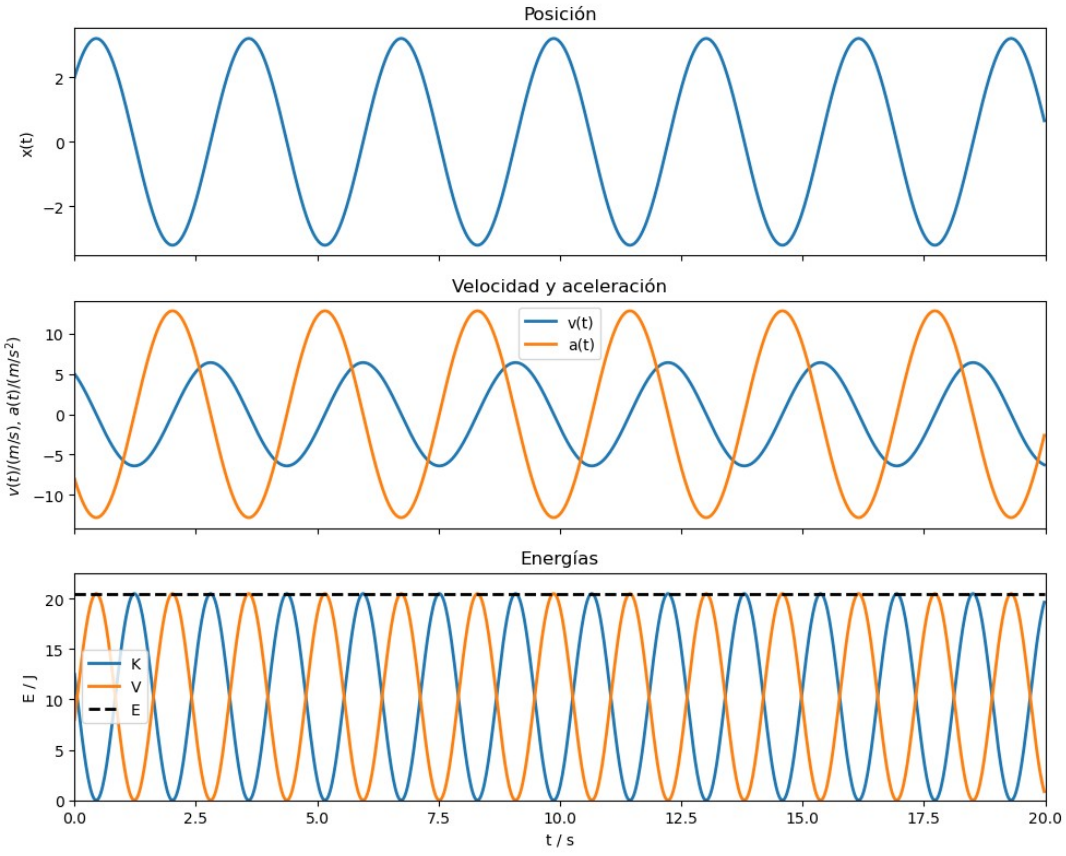


Figura 17: Gráfica de la nueva simulación.

¿Cuál es el valor máximo de la energía cinética y cuando se alcanza?

- `K_max` $\rightarrow$ 16,5 J. Se alcanza cuando la posición en $x = 0$, que es el punto de equilibrio. Coincide con los momentos de máxima velocidad que dependiendo de su dirección será **positiva o negativa**. ($v = \pm$ 5,66 m/s)

¿Cuál es el valor máximo de la energía potencial y cuando se alcanza?

- `U_max` $\rightarrow$ 8,0 J Se alcanza en los puntos de máxima elongación. Coincide con los momentos donde la velocidad es cero. ($x = \pm$ 2,83 m)

Energía total (E) ¿Esperabamos este resultado?

- Constanten en 16,5 J. Es lo que esperabamos, dado que es un oscilador amónico **sin amortiguamiento.**

2.2. Amortiguamiento porporcional a la velocidad.

Creamos una variable `(gamma = 0.5)` y modificamos, `(a = -omega**2 * x)` en `(a = -omega**2 * x - gamma * v**2)`.

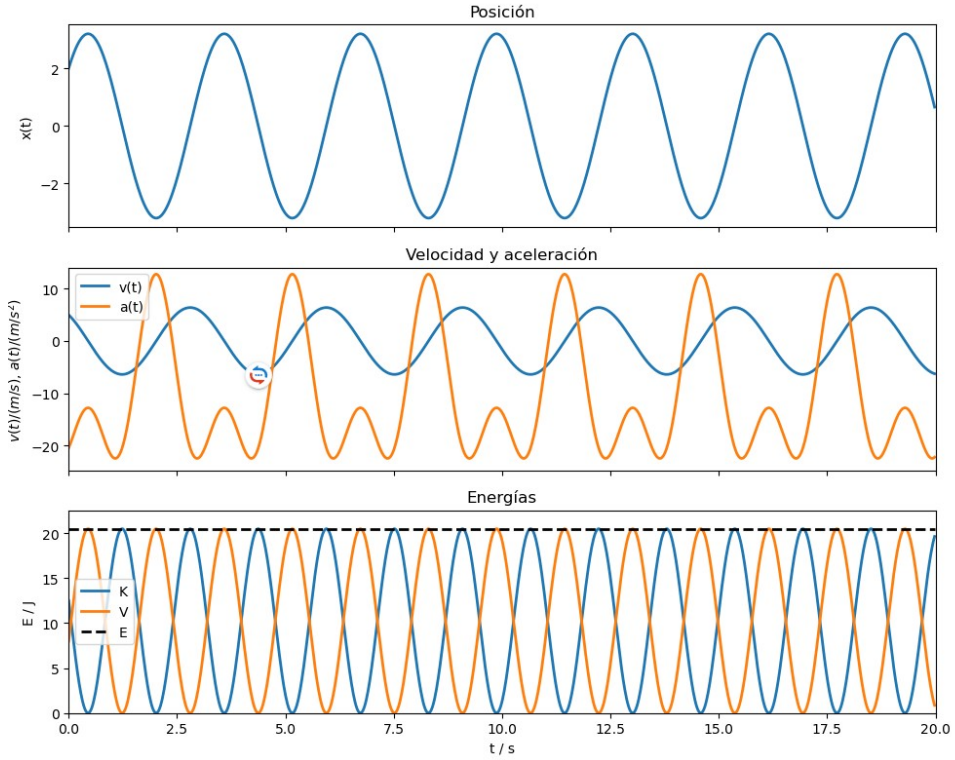


Figura 18: Gráfica con γ constante y $a = -\omega^2 \cdot x - \gamma \cdot v^2$.

3. Ejercicio 3: Pulso en una cuerda.

Un hombre está tendiendo la ropa en unas cuerdas y se da cuenta de que, al subir y bajar uno de los extremos de la cuerda se genera un pulso que se aleja de él. Este, al llegar al otro extremo de la pared, es reflejado de vuelta hacia el hombre. Para entender mejor el suceso, el hombre decide hacer una simulación en el ordenador.

1. Ejecuta la simulación. Una vez la ejecutes, cambia los parámetros del pulso inicial (`sigma`, `x0`...). Adjunta la animación resultante.

2. Modifica el pulso inicial (función `initial_pulse` en el código) para que en vez de un pulso a la izquierda, también haya otro pulso a la derecha. Debes simular el caso en el que este segundo pulso tiene amplitud positiva y el caso con amplitud negativa. Explica cómo interfieren las ondas en cada caso.

3. Modifica el impulso inicial para tal que sea una onda cuadrada (por ejemplo, `np.where(x >0.4) & (x <0.6), 1.0, 0.0)`. Adjunta una figura de tus resultados. Comenta el resultado.

3.1. Parámetros del pulso inicial.

Ajustamos el pulso inicial. `x0=0.5` y `sigma=0.09` (Se adjuntará esta y las demás animaciones en un archivo aparte.)

```python
# Pulso (gaussiano) inicial
def initial_pulse(x, x0=0.5, sigma=0.09):
    return np.exp(-((x - x0) ** 2) / (2 * sigma ** 2))
```

Figura 19: Modificación del pulso gaussiano.

Siendo nuestra gráfica:

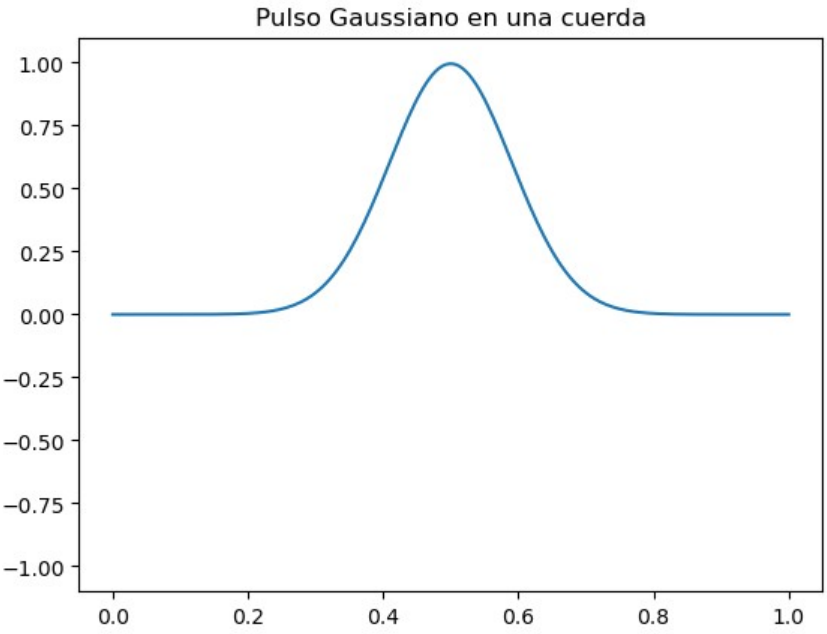


Figura 20: Pulso gaussiano.

3.2. Modificación del pulso inicial.

Modificamos la función `initial_pulse` e incluimos dos pulsos.

```
# Pulso (gaussiano) inicial
def initial_pulse(x, x0=0.3, x1=0.7, sigma=0.05, amplitude=1.0):
    pulse1 = np.exp(-((x - x0)**2)/(2*sigma**2))
    pulse2 = amplitude * np.exp(-((x - x1)**2)/(2*sigma**2))
    return pulse1 + pulse2
```

Figura 21: Modificación del pulso gaussiano.

Figura con la amplitud positiva.

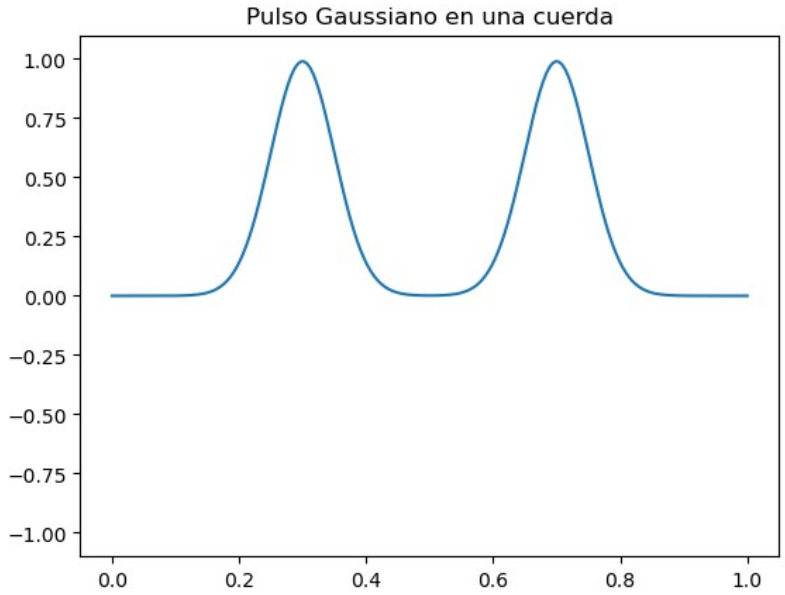


Figura 22: Modificación del pulso gaussiano con amplitud positiva.

Cuando el segundo pulso tiene amplitud positiva, las ondas se interfieren **constructivamente** cuando se encuentran en el centro. La amplitud resultante es la suma de ambas ondas.

Figura con la amplitud negativa.

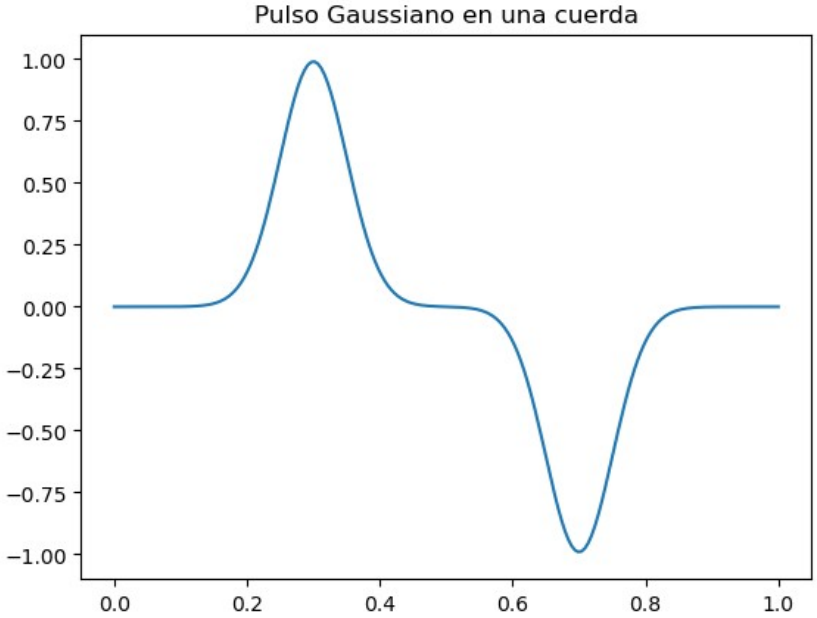


Figura 23: Modificación del pulso gaussiano con amplitud negativa.

Cuando el segundo pulso tiene amplitud negativa, las ondas interfieren **destructivamente** cuando se encuentran, cancelñandose parcial o totalmente dependiendiendo de la fase exacta.

3.3. Pulso inicial - onda cuadrada.

Modificamos el pulso inicial:

```python
# Pulso (gaussiano) inicial
def initial_pulse(x):
    return np.where((x > 0.4) & (x < 0.6), 1.0, 0.0)
```

Figura 24: Modificación del pulso gaussiano en onda cuadrada.

Donde obtenemos la siguiente gráfica.

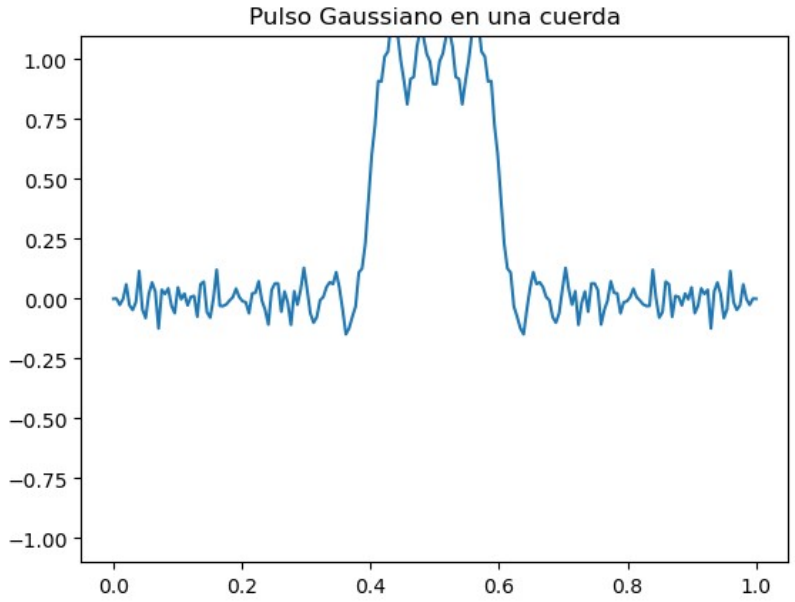


Figura 25: Modificación del pulso gaussiano en onda cuadrada.

Comentarios del resultado:

- La onda cuadrada muestra dispersión porque sus diferentes componentes de frecuencia viajan a velocidades ligeramente diferentes, además que se se descompone rápidamente en una serie de ondas más complejas debido a que contiene más frecuencias (componentes de Fourier).

4. Ejercicio 4: Ruptura de la velocidad del sonido.

El ejercicio 4 de esta simulación consiste en ejecutar el código de una simulación del sonido que emite un jet supersónico, el cual viaja a una velocidad constante de $1{,}5v_s$, donde v_s es la velocidad del sonido (Mach 1.5).

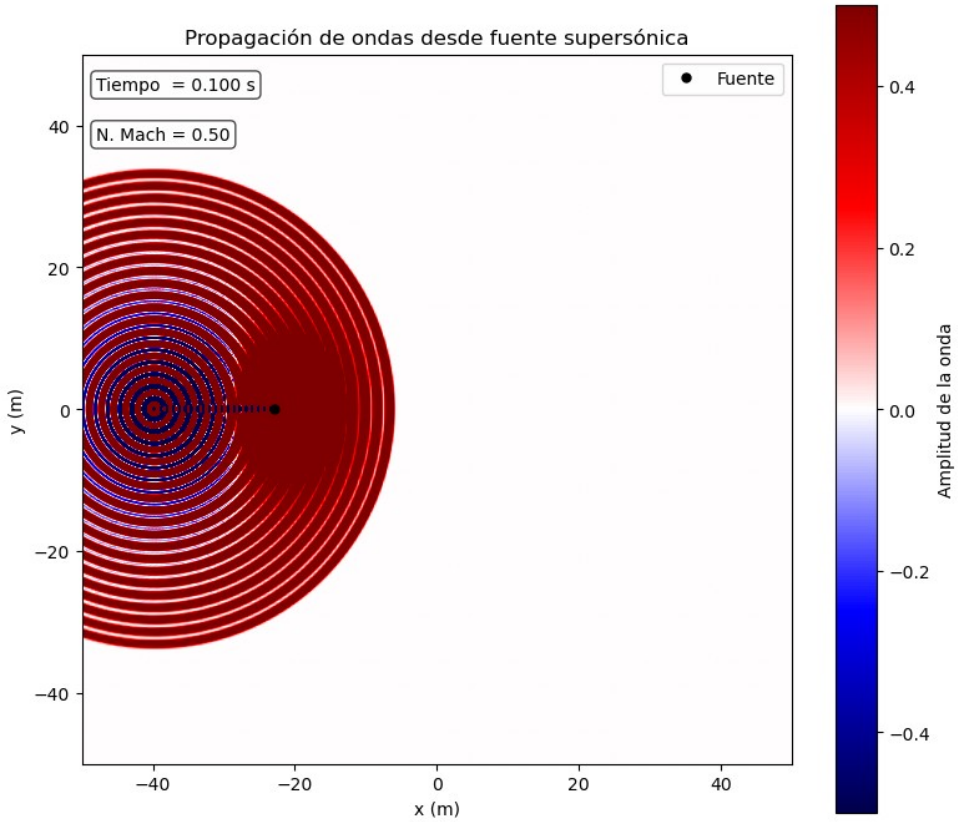
1. Modifica el código para que el jet viaje a $0{,}5v_s$, $0{,}75v_s$, $1{,}0v_s$ y $2{,}0v_s$. Adjunta las imágenes resultantes y comenta cada uno de los resultados. Concretamente: ¿Qué pasa cuando la velocidad del jet es la velocidad del sonido? ¿Qué pasa si la excedemos?

2. Aprovecha esta simulación para hacer una explicación del efecto Doppler. ¿Es válido el efecto Doppler cuando el objeto viaja a velocidades mayores a la velocidad del sonido? Argumenta tu respuesta con las fórmulas pertinentes.

3. Finalmente, realiza una última simulación donde la posición inicial es el origen de coordenadas y el jet está quieto, aunque emitiendo sonido. ¿Se recupera el resultado que esperabas?

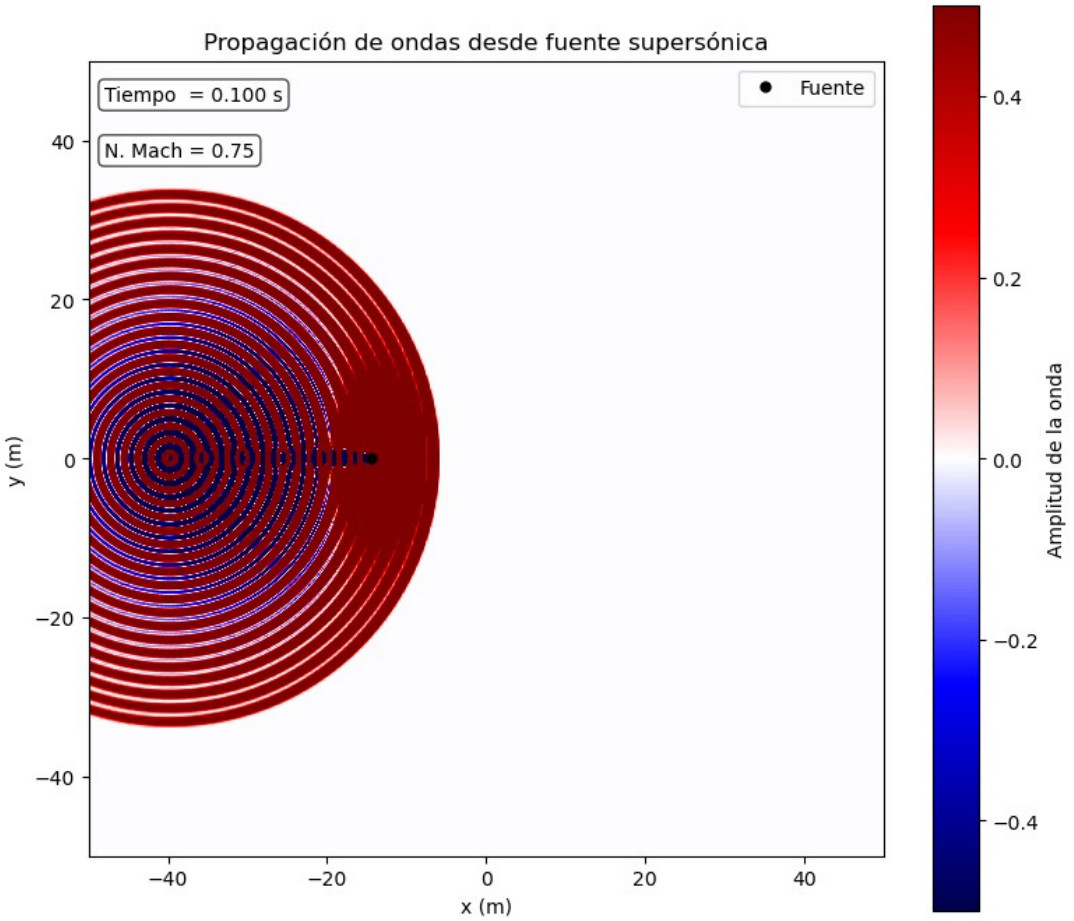
4.1. Velocidad Jet.

Gráfica del jet cuando `v_s = 0.5`



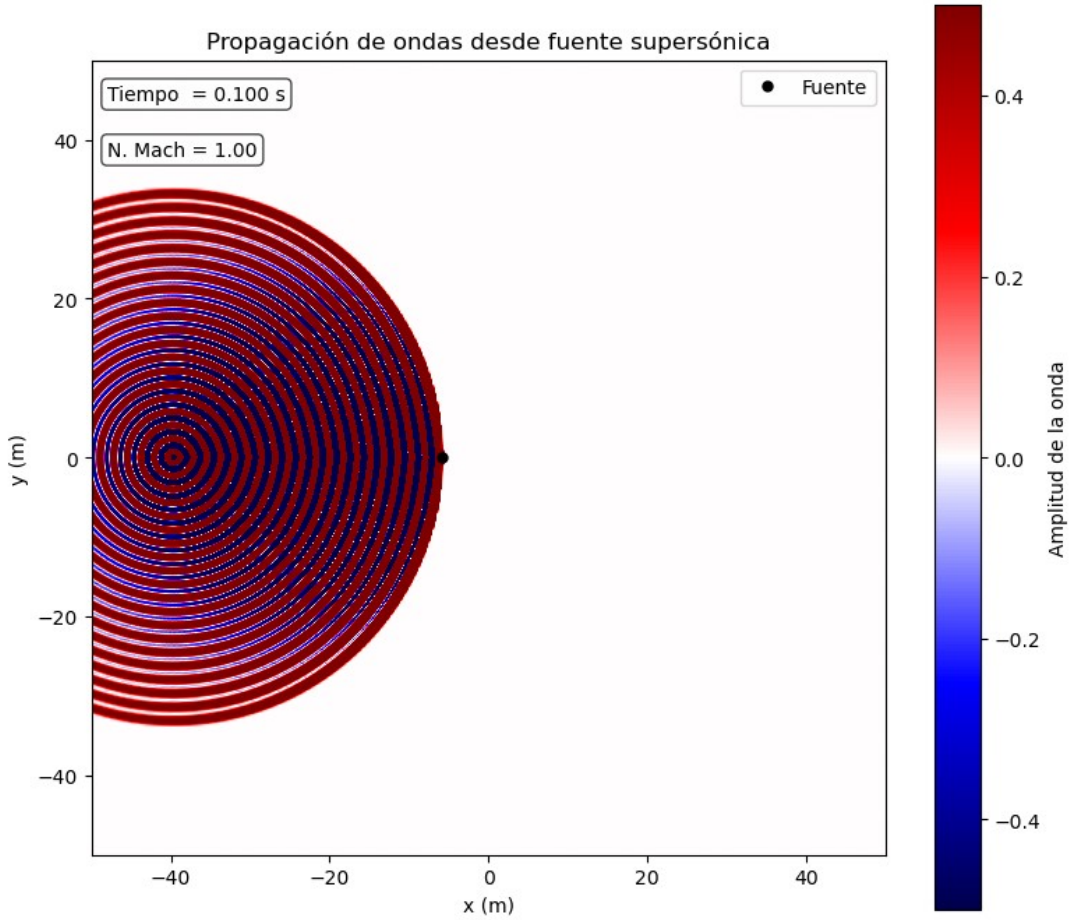
Cuando `v_s = 0.5` el jet se mueve **más lento que el sonido** (subsónico). La amplitud es mayor frente a la fuente (**efecto Doppler**) y menor detrás.

Gráfica del jet cuando `v_s = 0.75`



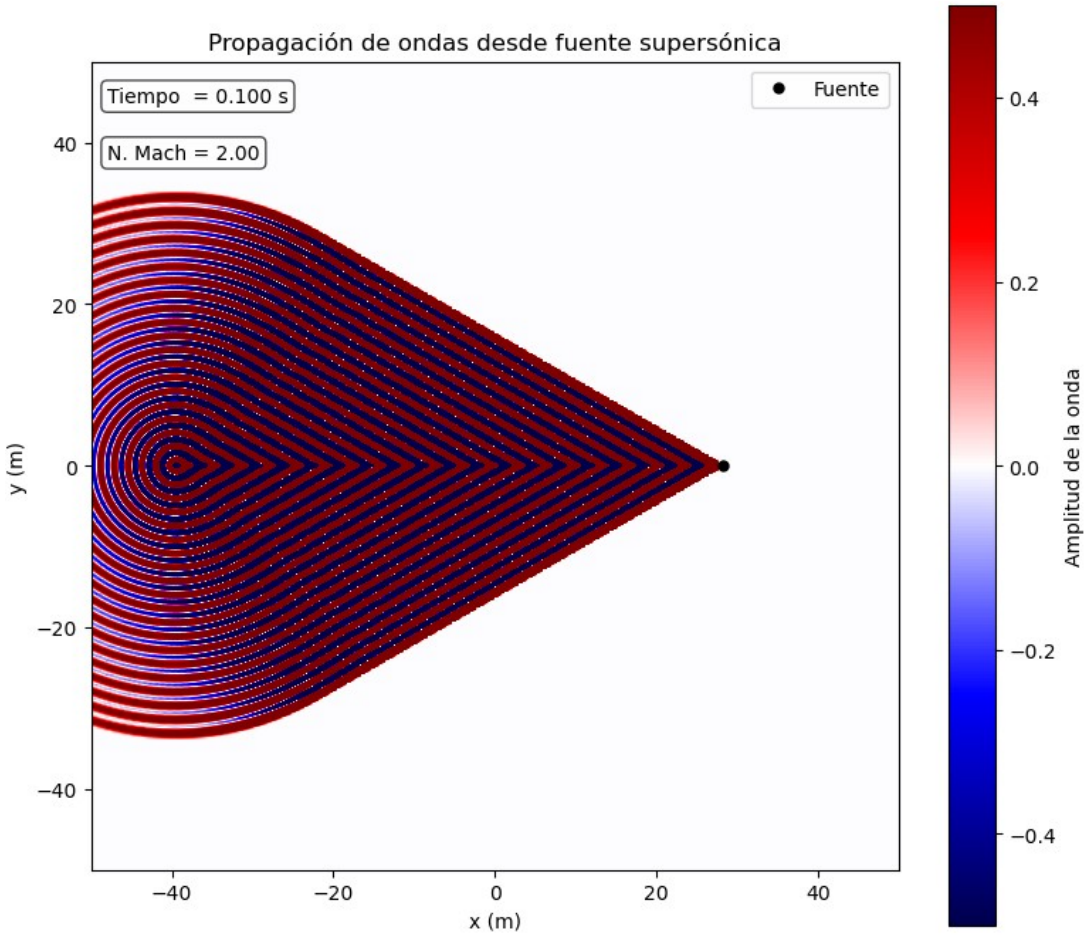
Cuando `v_s = 0.75` se mueve **más lento que el sonido** (subsónico), se comporta similar al punto pasado.

Gráfica del jet cuando `v_s = 1.0`



Cuando `v_s = 1.0` se mueve **a la misma velocidad que el sonido** (sónico). Las ondas emitidas se acumulan en el frente del jet, creando una interferencia constructiva en $x = 0$.

Gráfica del jet cuando `v_s = 2.0`



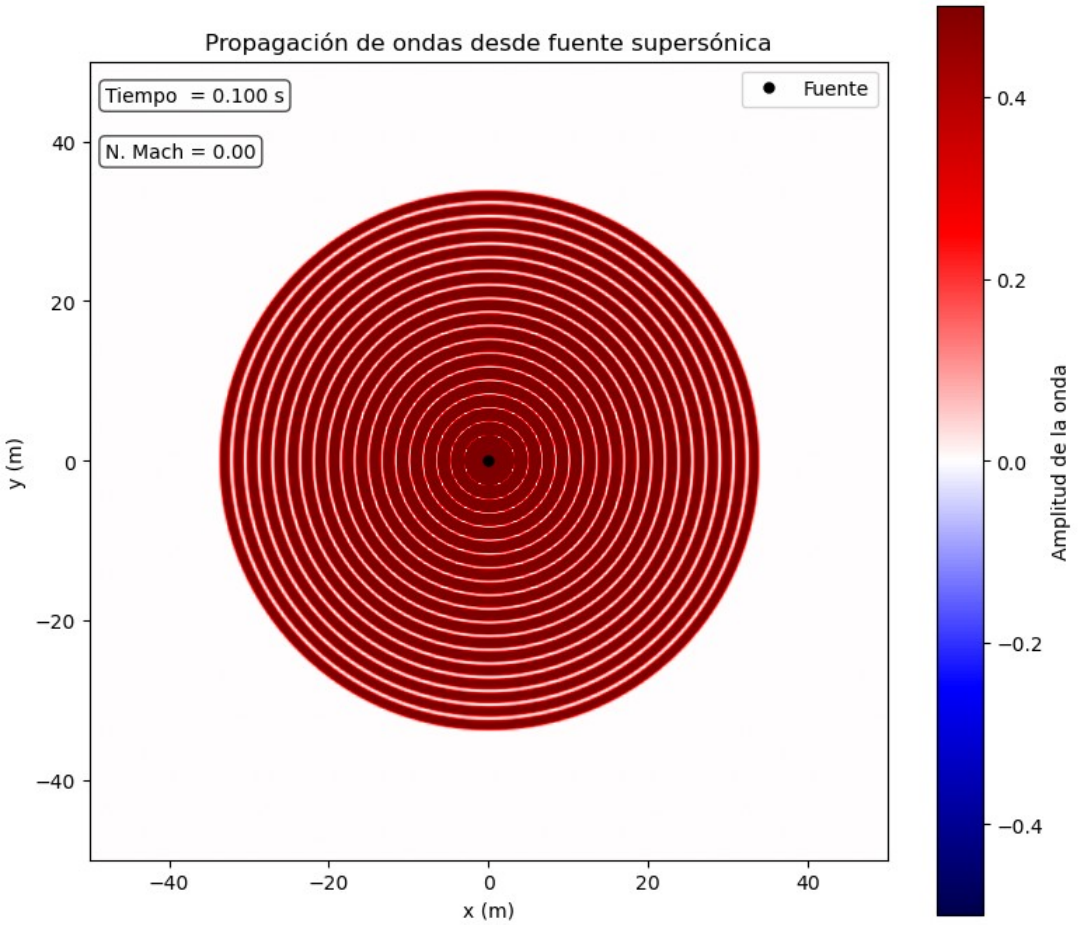
Cuando `v_s = 2.0` se mueve **más rapido que el sonido** (supersónico). LSe forma un cono de Mach (frente de choque).

4.2. Interpretación efecto Doppler.

Si el jet es más rápido que el sonido `v_s >C`, la frecuencia percibida puede ser negativa o indefinida amtes de que la fuente pase al observador, además, el observador no escucharía nada hasta que el jet haya pasado su posición, en ese instante cuando el jet pasa, se percibe un **boom sónico** que es una onda de choque, no una onda de sonido continua.

4.3. Posición inicial en el origen de las coordenadas.

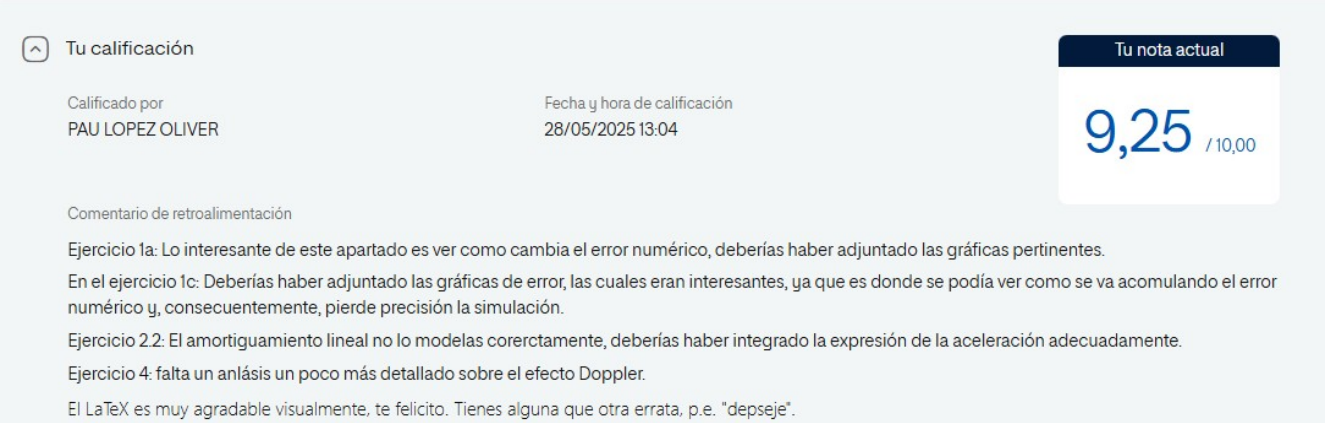
Modificamos la velocidad `v_s = 0` y el origen de coordenadas `x0 = 0`.



En este caso, las ondas deben propagrase de forma concéntrica, círculos simétricos alrededor del origen.

Si es lo que esperamos en la animación de este caso, dado que refleja como las ondas se expanden uniformemente, sin compresión o rarefacción en la dirección de movimiento.

5. Califiación/Observaciones.



Tu calificación

Tu nota actual

9,25 / 10,00

Calificado por
PAU LOPEZ OLIVER

Fecha y hora de calificación
28/05/2025 13:04

Comentario de retroalimentación

Ejercicio 1a: Lo interesante de este apartado es ver como cambia el error numérico, deberías haber adjuntado las gráficas pertinentes.

En el ejercicio 1c: Deberías haber adjuntado las gráficas de error, las cuales eran interesantes, ya que es donde se podía ver como se va acomulando el error numérico y, consecuentemente, pierde precisión la simulación.

Ejercicio 2.2: El amortiguamiento lineal no lo modelas corerctamente, deberías haber integrado la expresión de la aceleración adecuadamente.

Ejercicio 4: falta un anlásis un poco más detallado sobre el efecto Doppler.

El LaTeX es muy agradable visualmente, te felicito. Tienes alguna que otra errata, p.e. "depseje".